

LAPORAN PRAKTIKUM

STRUKTUR DATA

Dosen Pengampu : Dr. Wahyudi, S.T, M.T



Disusun Oleh:
Annafi Al Ghifari
2511533013

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan tugas mata kuliah *Struktur Data* ini dengan baik dan tepat waktu.

Laporan/tugas ini disusun sebagai salah satu bentuk pemenuhan kewajiban akademik pada mata kuliah Struktur Data. Dalam penyusunannya, penulis berusaha memahami konsep-konsep dasar yang berkaitan dengan struktur data, seperti pengelolaan data menggunakan array, stack, queue, serta implementasi dalam pemrograman. Diharapkan melalui tugas ini, penulis dapat meningkatkan pemahaman serta kemampuan dalam menerapkan konsep struktur data secara praktis.

Penulis menyadari bahwa dalam penyusunan tugas ini masih terdapat banyak kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data serta semua pihak yang telah membantu dalam proses penyusunan tugas ini.

Semoga laporan ini dapat memberikan manfaat bagi penulis khususnya dan bagi pembaca pada umumnya.

Padang, 21 Mei 2026

Annafi Al Ghifari

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
BAB II PEMBAHASAN.....	2
2.1 Program Dan Kodingan	2
2.2 Tugas	11
BAB III KESIMPULAN	20
3.1 Ringkasan.....	20
3.2 Saran	20
DAFTAR PUSTAKA	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia komputer dan teknologi informasi, pengolahan data merupakan hal yang sangat penting. Data yang tersusun dengan rapi dan teratur akan memudahkan proses pencarian, pengelompokan, serta analisis informasi. Oleh karena itu, diperlukan suatu metode atau algoritma yang dapat digunakan untuk mengurutkan data secara efektif dan efisien. Proses pengurutan data tersebut dikenal dengan istilah sorting.

Algoritma sorting merupakan salah satu materi dasar dalam pemrograman dan struktur data yang wajib dipahami oleh mahasiswa informatika. Dengan memahami algoritma pengurutan, seseorang dapat meningkatkan kemampuan dalam menyusun logika program serta menyelesaikan berbagai permasalahan yang berkaitan dengan pengolahan data. Terdapat berbagai jenis algoritma pengurutan, namun beberapa metode dasar yang paling sering dipelajari adalah Insertion Sort, Selection Sort, dan Bubble Sort.

Insertion Sort adalah metode pengurutan dengan cara menyisipkan data ke posisi yang sesuai pada bagian data yang telah teratur. Selection Sort bekerja dengan memilih nilai terkecil atau terbesar dari sekumpulan data untuk ditempatkan pada posisi yang benar. Sementara itu, Bubble Sort melakukan pengurutan dengan membandingkan dua data yang berdekatan secara berulang hingga data tersusun dengan benar. Ketiga metode tersebut memiliki cara kerja, tingkat efisiensi, serta kelebihan dan kekurangan yang berbeda.

BAB II

PEMBAHASAN

1. Program Dan Kodingan

1. BubleSort_2511533013

```
2. package pekan7_2511533013;
3. public class BubleSort_2511533013 {
4.     public static void bubbleSort_2511533013(int[] arr) {
5.         int n_3013 = arr.length;
6.         for (int i_3013 = 0; i_3013 < n_3013; i_3013++) {
7.             for (int j_3013 = 0; j_3013 < n_3013 - i_3013 - 1;
8.                 j_3013++) {
9.                 if (arr[j_3013] > arr[j_3013 + 1]) {
10.                     int temp = arr[j_3013];
11.                     arr[j_3013] = arr[j_3013 + 1];
12.                     arr[j_3013 + 1] = temp;
13.                     // System.out.println("data:" + arr[j_3013] + " "
14.                     + arr[j_3013 + 1]);
15.                 }
16.             }
17.         }
18.     }
19.     public static void main (String[] args) {
20.         int arr[] = {23, 78, 45, 8, 32, 56, 1};
21.         int n_3013 = arr.length;
22.         System.out.println("array yang belum terurut:\n");
23.         for (int i_3013 = 0; i_3013 < n_3013; i_3013++) {
24.             System.out.print(arr[i_3013] + " ");
25.             System.out.println("");
26.             // minMaxSelectionSort_2511533013(arr, n);
27.         }
28.         bubbleSort_2511533013(arr);
29.         System.out.println("array yang terurut menggunakan
30.         Bubble Sort:");
31.         for (int i_3013 = 0; i_3013 < n_3013; i_3013++) {
32.             System.out.print(arr[i_3013] + " ");
33.             System.out.println("");
34.         }
35.     }
36. }
```

OUTPUT:

```
<terminated> BubleSort_2511533013 [Java Application] C:\Users\anna
array yang belum terurut:
23
78
45
8
32
56
1
array yang terurut menggunakan Bubble Sort:
1
8
23
32
45
56
78
```

Penjelasan :

1. **Method bubbleSort_2511533013**

Method ini digunakan untuk mengurutkan array menggunakan metode Bubble Sort.

2. **Variabel n_3013**

Digunakan untuk menyimpan jumlah elemen dalam array menggunakan arr.length.

3. **Perulangan For Pertama**

Digunakan untuk menentukan jumlah proses pengurutan (pass) pada Bubble Sort.

4. **Perulangan For Kedua**

Digunakan untuk membandingkan elemen yang berdekatan dalam array.

5. **Kondisi If**

Digunakan untuk mengecek apakah elemen kiri lebih besar dari elemen kanan.

6. **Variabel temp**

Digunakan sebagai variabel sementara saat proses pertukaran data.

7. **Proses Pertukaran Data**

Data ditukar agar posisi elemen menjadi urut dari kecil ke besar.

8. Method main

Merupakan titik awal eksekusi program.

2. SelectionSort_2511533013

```
3. package pekan7_2511533013;
4.
5. public class SelectionSort_2511533013 {
6.     public static void SelectionSort_2511533013(int[] arr) {
7.         int n_3013 = arr.length;
8.         for (int i_3013 = 0; i_3013 < n_3013; i_3013++) {
9.             int minIndex = i_3013;
10.            for (int j = i_3013 + 1; j < n_3013; j++) {
11.                if (arr[j] < arr[minIndex]) {
12.                    minIndex = j;
13.                }
14.            }
15.            int temp = arr[i_3013];
16.            arr[i_3013] = arr[minIndex];
17.            arr[minIndex] = temp;
18.        }
19.    }
20.    public static void main(String[] args) {
21.        int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
22.        int n_3013 = arr.length;
23.        System.out.println("array yang belum terurut:\n");
24.        for (int i_3013 = 0; i_3013 < n_3013; i_3013++) {
25.            System.out.print(arr[i_3013] + " ");
26.            System.out.println("");
27.            SelectionSort_2511533013(arr);
28.        }
29.        System.out.println("array yang terurut:\n");
30.        for (int i_3013 = 0; i_3013 < n_3013; i_3013++) {
31.            System.out.print(arr[i_3013] + " ");
32.            System.out.println("");
33.        }
34.    }
35. }
```

OUTPUT:

```
<terminated> SelectionSort_2511533013 [Java Application] C:\Users\annaf
array yang belum terurut:

23
8
23
32
45
56
78
array yang terurut:

1
8
23
32
45
56
78
```

Penjelasan:

1. **Method SelectionSort_2511533013**

Method ini digunakan untuk mengurutkan array menggunakan metode Selection Sort.

2. **Variabel n_3013**

Digunakan untuk menyimpan jumlah elemen array.

3. **Variabel minIndex**

Digunakan untuk menyimpan indeks nilai terkecil sementara.

4. **Perulangan For Pertama**

Digunakan untuk menentukan posisi data yang akan diisi nilai terkecil.

5. **Perulangan For Kedua**

Digunakan untuk mencari nilai terkecil dalam array.

6. **Kondisi If**

Digunakan untuk membandingkan nilai array dengan nilai minimum.

7. **Proses Pertukaran Data**

Digunakan untuk menukar nilai terkecil dengan posisi awal.

8. **Method main**

Merupakan titik awal eksekusi program.

3. InsertionSortGUI_2511533013

```
4. package pekan7_2511533013;
5.
6. import java.awt.BorderLayout;
7. import java.awt.Color;
8. import java.awt.Dimension;
9. import java.awt.EventQueue;
10. import java.awt.Font;
11.
12. import javax.swing.BorderFactory;
13. import javax.swing.JButton;
14. import javax.swing.JFrame;
15. import javax.swing.JLabel;
16. import javax.swing.JOptionPane;
17. import javax.swing.JPanel;
18. import javax.swing.JScrollPane;
19. import javax.swing.JTextArea;
20. import javax.swing.JTextField;
21. import javax.swing.SwingConstants;
22. import javax.swing.border.EmptyBorder;
23.
24. public class insertionSortGUI_2511533013 extends JFrame {
25.
26.     private static final long serialVersionUID = 1L;
27.     private int[] array;
28.     private JPanel[] labelArray;
29.     private JButton stepButton, resetButton, setButton;
30.     private JTextField inputField;
31.     private JPanel panelArray;
32.     private JTextArea stepArea;
33.
34.     private int i = 1, j;
35.     private boolean sorting = false;
36.     private int stepCount = 1;
37.     /**
38.      * Launch the application.
39.      */
40.     public static void main(String[] args) {
41.         EventQueue.invokeLater(new Runnable() {
42.             public void run() {
43.                 try {
44.                     insertionSortGUI_2511533013 frame
45. = new insertionSortGUI_2511533013();
46.                     frame.setVisible(true);
47.                 } catch (Exception e) {
48.                     e.printStackTrace();
49.                 }
50.             }
51.         });
52.     }
53.     /**
```

```

54.      * Create the frame.
55.      */
56.      public insertionSortGUI_2511533013() {
57.          setTitle("Insertion Sort Langkah per Langkah");
58.          setSize(750, 400);
59.          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
60.          setLocationRelativeTo(null);
61.          setLayout(new BorderLayout());
62.
63.          //      Panel input
64.          JPanel inputPanel = new JPanel (new FlowLayout());
65.          inputField = new JTextField(30);
66.          setButton = new JButton("Set Array");
67.          inputPanel.add(new JLabel ("Masukkan angka (pisahkan
        dengan koma) :"));
68.          //      Panel array visual
69.          panelArray = new JPanel();
70.          panelArray.setLayout(new FlowLayout());
71.          //      Panel kontrol
72.          JPanel controlPanel = new JPanel ();
73.          stepButton = new JButton ("Langkah Selanjutnya");
74.          resetButton = new JButton ("Reset");
75.          stepButton.setEnabled(false);
76.          controlPanel.add(stepButton);
77.          controlPanel.add(resetButton);
78.          //      Area teks untuk log langkah-langkah
79.          stepArea = new JTextArea(8, 60);
80.          stepArea.setEditable(false);
81.          stepArea.setFont (new Font("Monospaced", Font.PLAIN,
        14));
82.          JScrollPane scrollPane = new JScrollPane(stepArea);
83.          //      Tambahkan panel ke frame
84.          add(inputPanel, BorderLayout.NORTH);
85.          add(panelArray, BorderLayout.CENTER);
86.          add(controlPanel, BorderLayout.SOUTH);
87.          add(scrollPane, BorderLayout.EAST);
88.          //      Event Set Array
89.          setButton.addActionListener(e-> setArrayFromInput());
90.          //      Event Langkah Selanjutnya
91.          stepButton.addActionListener(e-> performStep());
92.          //      Event Reset
93.          resetButton.addActionListener(e-> resetSorting());
94.      }
95.
96.      private void reset() {
97.
98.          // Mengosongkan input field
99.          inputField.setText("");
100.
101.          // Menghapus seluruh komponen pada panel array
102.          panelArray.removeAll();
103.
104.          // Memperbarui tampilan panel

```

```

105.         panelArray.revalidate();
106.         panelArray.repaint();
107.
108.         // Mengosongkan area langkah sorting
109.         stepArea.setText("");
110.
111.         // Menonaktifkan tombol step
112.         stepButton.setEnabled(false);
113.
114.         // Menghentikan proses sorting
115.         sorting = false;
116.
117.         // Mengembalikan indeks awal insertion sort
118.         i = 1;
119.
120.         // Mengatur ulang jumlah langkah
121.         stepCount = 1;
122.     }
123.
124.     private void performStep() {
125.         if (i < array.length && sorting) {
126.
127.             // Menyimpan elemen yang akan disisipkan
128.             int key = array[i];
129.
130.             // Menentukan indeks sebelumnya
131.             j = i - 1;
132.
133.             // Membuat log langkah sorting
134.             StringBuilder stepLog = new StringBuilder();
135.             stepLog.append("Langkah ")
136.                 .append(stepCount)
137.                 .append(": Memasukkan ")
138.                 .append(key)
139.                 .append("\n");
140.
141.             // Menggeser elemen yang lebih besar dari key
142.             while (j >= 0 && array[j] > key) {
143.                 array[j + 1] = array[j];
144.                 j--;
145.             }
146.
147.             // Menempatkan key pada posisi yang benar
148.             array[j + 1] = key;
149.
150.             // Memperbarui tampilan array
151.             updateLabels();
152.
153.             // Menampilkan hasil setiap langkah
154.             stepLog.append("Hasil: ")
155.                 .append(arrayToString(array))
156.                 .append("\n\n");
157.

```

```

158.         stepArea.append(stepLog.toString());
159.
160.         // Pindah ke elemen berikutnya
161.         i++;
162.         stepCount++;
163.
164.         // Jika sorting selesai
165.         if (i == array.length) {
166.             sorting = false;
167.
168.             // Menonaktifkan tombol step
169.             stepButton.setEnabled(false);
170.
171.             // Menampilkan pesan selesai
172.             JOptionPane.showMessageDialog(this, "Sorting
selesai!");
173.         }
174.     }
175. }
176.     private void updateLabels() {
177.
178.         // Melakukan perulangan untuk seluruh elemen array
179.         for (int k = 0; k < array.length; k++) {
180.
181.             // Mengubah isi label sesuai nilai array
182.             labelArray[k].setText(String.valueOf(array[k]));
183.         }
184.     }
185.
186.     private String arrayToString(int[] arr) {
187.
188.         // Membuat objek StringBuilder
189.         StringBuilder sb = new StringBuilder();
190.
191.         // Perulangan untuk menggabungkan isi array menjadi
string
192.         for (int k = 0; k < arr.length; k++) {
193.
194.             // Menambahkan elemen array ke StringBuilder
195.             sb.append(arr[k]);
196.
197.             // Menambahkan koma jika bukan elemen terakhir
198.             if (k < arr.length - 1)
199.                 sb.append(", ");
200.         }
201.
202.         // Mengembalikan hasil string array
203.         return sb.toString();
204.     }
205.
206.     private void setArrayFromInput() {
207.         String text = inputField.getText().trim();
208.         if (text.isEmpty()) {

```

```

209.         return;
210.     }
211.     String[] parts = text.split(",");
212.     array = new int[parts.length];
213.     try {
214.         for (int k = 0; k < parts.length;
215.             k++) {
216.             array[k] =
217.                 Integer.parseInt(parts[k].trim());
218.         } catch (NumberFormatException e) {
219.             JOptionPane.showMessageDialog(this, "Masukkan hanya angka
220.                 yang dipisahkan " + "dengan koma", JOptionPane.ERROR_MESSAGE);
221.             return;
222.         }
223.         i = 1;
224.         stepCount = 1;
225.         sorting = true;
226.         stepArea.setText("");
227.         panelArray.removeAll();
228.         labelArray = new JLabel[array.length];
229.         for (int k = 0; k < array.length; k++) {
230.             labelArray[k] = new
231.                 JLabel(String.valueOf(array[k]));
232.             labelArray[k].setFont(new Font("Arial",
233.                 Font.BOLD, 24));
234.             labelArray[k].setBorder(BorderFactory.createLineBorder(Color
235.                 .BLACK));
236.             labelArray[k].setPreferredSize(new
237.                 Dimension(50, 50));
238.             labelArray[k].setHorizontalAlignment(SwingConstants.CENTER);
239.             panelArray.add(labelArray[k]);
240.         }
241.         panelArray.revalidate();
242.         panelArray.repaint();
243.     }
244.     updateArrayDisplay();
245.     stepButton.setEnabled(true);
246.     sorting = true;
247.     i = 1;
248.     j = i - 1;
249.     stepCount = 1;
250.     stepArea.setText("");
251. }

```

2. Tugas

1. Mahasiswa_2511533013 (Sebagai Kelas ADT)

```
3. package pekan7_2511533013;
4.
5. public class Mahasiswa_2511533013 {
6.     private String nama_3013;
7.     private String nim_3013;
8.     private String prodi_3013;
9.
10.    public Mahasiswa_2511533013(String nama_3013, String
nim_3013, String prodi_3013) {
11.        this.nama_3013 = nama_3013;
12.        this.nim_3013 = nim_3013;
13.        this.prodi_3013 = prodi_3013;
14.    }
15.
16.    public String getNama_3013() {
17.        return nama_3013;
18.    }
19.
20.    public void setNama_3013(String nama_3013) {
21.        this.nama_3013 = nama_3013;
22.    }
23.
24.    public String getNim_3013() {
25.        return nim_3013;
26.    }
27.
28.    public void setNim_3013(String nim_3013) {
29.        this.nim_3013 = nim_3013;
30.    }
31.
32.    public String getProdi_3013() {
33.        return prodi_3013;
34.    }
35.
36.    public void setProdi_3013(String prodi_3013) {
37.        this.prodi_3013 = prodi_3013;
38.    }
39.
40.    @Override
41.    public String toString() {
42.        return nama_3013 + " - " + nim_3013 + " - " +
prodi_3013;
43.    }
44. }
45.
```

OUTPUT:

Penjelasan :

1. Class insertionsortGUI_2511533013

Class ini digunakan untuk membuat program GUI Insertion Sort menggunakan Java Swing.

2. Variabel array

Digunakan untuk menyimpan data angka yang akan diurutkan.

3. JButton

Digunakan untuk membuat tombol Set Array, Langkah Selanjutnya, dan Reset.

4. JTextField inputField

Digunakan untuk memasukkan angka ke dalam program.

5. Method setArrayFromInput()

Digunakan untuk mengambil input angka dari pengguna lalu mengubahnya menjadi array integer.

6. Method performStep()

Digunakan untuk menjalankan proses Insertion Sort langkah demi langkah.

7. Variabel key

Digunakan untuk menyimpan elemen yang akan disisipkan ke posisi yang benar.

8. Perulangan While

Digunakan untuk menggeser elemen yang lebih besar dari key ke posisi kanan.

9. Method updateLabels()

Digunakan untuk memperbarui tampilan array pada GUI.

10. Method arrayToString()

Digunakan untuk mengubah isi array menjadi bentuk string.

11. JTextArea stepArea

Digunakan untuk menampilkan proses sorting langkah demi langkah.

12. Method main

Merupakan titik awal eksekusi program GUI.

1. SortingGUI_2511533013

```
1. package pekan7_2511533013;
2.
3. import javax.swing.*;
4. import javax.swing.table.DefaultTableModel;
5. import java.awt.*;
6. import java.awt.event.*;
7. import java.util.ArrayList;
8.
9. public class SortingGUI_2511533013 extends JFrame {
10.
11.     JTextField txtNama_3013, txtNim_3013, txtProdi_3013;
12.     JButton btnTambah_3013, btnHapus_3013, btnSort_3013;
13.     JComboBox<String> cbSorting_3013;
14.     JTable table_3013;
15.     JTextArea areaProses_3013;
16.
17.     DefaultTableModel model_3013;
18.
19.     ArrayList<Mahasiswa_2511533013> listMahasiswa_3013 = new
        ArrayList<>();
20.
21.     public SortingGUI_2511533013() {
22.         setTitle("Sorting Mahasiswa GUI");
23.         setSize(800, 600);
24.         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
25.         setLocationRelativeTo(null);
26.
27.         setLayout(new BorderLayout());
28.
29.         JPanel panelInput_3013 = new JPanel(new GridLayout(4,
30.             2));
31.         panelInput_3013.add(new JLabel("Nama"));
32.         txtNama_3013 = new JTextField();
33.         panelInput_3013.add(txtNama_3013);
34.
35.         panelInput_3013.add(new JLabel("NIM"));
36.         txtNim_3013 = new JTextField();
37.         panelInput_3013.add(txtNim_3013);
38.
39.         panelInput_3013.add(new JLabel("Program Studi"));
40.         txtProdi_3013 = new JTextField();
41.         panelInput_3013.add(txtProdi_3013);
42.
43.         cbSorting_3013 = new JComboBox<>();
44.         cbSorting_3013.addItem("Insertion Sort");
45.         cbSorting_3013.addItem("Selection Sort");
46.         cbSorting_3013.addItem("Bubble Sort");
47.
48.         panelInput_3013.add(cbSorting_3013);
49.     }
```



```

50.         JPanel panelButton_3013 = new JPanel();
51.
52.         btnTambah_3013 = new JButton("Tambah Data");
53.         btnHapus_3013 = new JButton("Hapus Data");
54.         btnSort_3013 = new JButton("Mulai Sorting");
55.
56.         panelButton_3013.add(btnTambah_3013);
57.         panelButton_3013.add(btnHapus_3013);
58.         panelButton_3013.add(btnSort_3013);
59.
60.         model_3013 = new DefaultTableModel();
61.         model_3013.addColumn("Nama");
62.         model_3013.addColumn("NIM");
63.         model_3013.addColumn("Program Studi");
64.
65.         table_3013 = new JTable(model_3013);
66.
67.         areaProses_3013 = new JTextArea();
68.         areaProses_3013.setEditable(false);
69.
70.         add(panelInput_3013, BorderLayout.NORTH);
71.         add(new JScrollPane(table_3013), BorderLayout.CENTER);
72.         add(panelButton_3013, BorderLayout.SOUTH);
73.         add(new JScrollPane(areaProses_3013),
BorderLayout.EAST);
74.
75.         btnTambah_3013.addActionListener(new ActionListener() {
76.             @Override
77.             public void actionPerformed(ActionEvent e) {
78.                 tambahData_3013();
79.             }
80.         });
81.
82.         btnHapus_3013.addActionListener(new ActionListener() {
83.             @Override
84.             public void actionPerformed(ActionEvent e) {
85.                 hapusData_3013();
86.             }
87.         });
88.
89.         btnSort_3013.addActionListener(new ActionListener() {
90.             @Override
91.             public void actionPerformed(ActionEvent e) {
92.                 sortingData_3013();
93.             }
94.         });
95.     }
96.
97.     public void tambahData_3013() {
98.         String nama_3013 = txtNama_3013.getText();
99.         String nim_3013 = txtNim_3013.getText();
100.         String prodi_3013 = txtProdi_3013.getText();
101.

```

```

102.         Mahasiswa_2511533013 mhs_3013 = new
        Mahasiswa_2511533013(nama_3013, nim_3013, prodi_3013);
103.
104.         listMahasiswa_3013.add(mhs_3013);
105.
106.         model_3013.addRow(new Object[]{nama_3013, nim_3013,
        prodi_3013});
107.
108.         txtNama_3013.setText("");
109.         txtNim_3013.setText("");
110.         txtProdi_3013.setText("");
111.     }
112.
113.     public void hapusData_3013() {
114.         int baris_3013 = table_3013.getSelectedRow();
115.
116.         if (baris_3013 >= 0) {
117.             listMahasiswa_3013.remove(baris_3013);
118.             model_3013.removeRow(baris_3013);
119.         }
120.     }
121.
122.     public void sortingData_3013() {
123.         areaProses_3013.setText("");
124.
125.         String pilihan_3013 =
        cbSorting_3013.getSelectedItem().toString();
126.
127.         if (pilihan_3013.equals("Insertion Sort")) {
128.             insertionSort_3013();
129.         } else if (pilihan_3013.equals("Selection Sort")) {
130.             selectionSort_3013();
131.         } else {
132.             bubbleSort_3013();
133.         }
134.
135.         tampilTable_3013();
136.     }
137.
138.     public void tampilTable_3013() {
139.         model_3013.setRowCount(0);
140.
141.         for (Mahasiswa_2511533013 mhs_3013 :
        listMahasiswa_3013) {
142.             model_3013.addRow(new Object[]{
143.                 mhs_3013.getNama_3013(),
144.                 mhs_3013.getNim_3013(),
145.                 mhs_3013.getProdi_3013()
146.             });
147.         }
148.     }
149.
150.     public void insertionSort_3013() {

```

```

151.         areaProses_3013.append("=== INSERTION SORT ===\n");
152.
153.         for (int i_3013 = 1; i_3013 <
154.             listMahasiswa_3013.size(); i_3013++) {
155.             Mahasiswa_2511533013 key_3013 =
156.                 listMahasiswa_3013.get(i_3013);
157.             int j_3013 = i_3013 - 1;
158.             while (j_3013 >= 0 &&
159.                 listMahasiswa_3013.get(j_3013)
160.                     .getNama_3013()
161.                     .compareToIgnoreCase(key_3013.getNama_3013()) > 0) {
162.                 listMahasiswa_3013.set(j_3013 + 1,
163.                     listMahasiswa_3013.get(j_3013));
164.                 j_3013--;
165.             }
166.             listMahasiswa_3013.set(j_3013 + 1, key_3013);
167.             areaProses_3013.append("Langkah " + i_3013 + " :
168. " + tampilNama_3013() + "\n");
169.         }
170.     }
171. }
172.
173. public void selectionSort_3013() {
174.     areaProses_3013.append("=== SELECTION SORT ===\n");
175.
176.     for (int i_3013 = 0; i_3013 <
177.         listMahasiswa_3013.size() - 1; i_3013++) {
178.         int min_3013 = i_3013;
179.
180.         for (int j_3013 = i_3013 + 1; j_3013 <
181.             listMahasiswa_3013.size(); j_3013++) {
182.             if (listMahasiswa_3013.get(j_3013)
183.                 .getNama_3013()
184.                 .compareToIgnoreCase(
185.                     listMahasiswa_3013.get(min_3013).getNama_3013()) < 0) {
186.                 min_3013 = j_3013;
187.             }
188.         }
189.     }
190.
191.     Mahasiswa_2511533013 temp_3013 =
192.         listMahasiswa_3013.get(i_3013);
193.     listMahasiswa_3013.set(i_3013,
194.         listMahasiswa_3013.get(min_3013));
195.     listMahasiswa_3013.set(min_3013, temp_3013);

```

```

194.
195.         areaProses_3013.append("Pass " + (i_3013 + 1) +
" : " + tampilNama_3013() + "\n");
196.     }
197. }
198.
199.     public void bubbleSort_3013() {
200.         areaProses_3013.append("=== BUBBLE SORT ===\n");
201.
202.         for (int i_3013 = 0; i_3013 <
listMahasiswa_3013.size() - 1; i_3013++) {
203.
204.             for (int j_3013 = 0; j_3013 <
listMahasiswa_3013.size() - i_3013 - 1; j_3013++) {
205.
206.                 if (listMahasiswa_3013.get(j_3013)
207.                     .getNama_3013()
208.                     .compareToIgnoreCase(
209. listMahasiswa_3013.get(j_3013 + 1).getNama_3013()) > 0) {
210.
211.                     Mahasiswa_2511533013 temp_3013 =
listMahasiswa_3013.get(j_3013);
212.                     listMahasiswa_3013.set(j_3013,
listMahasiswa_3013.get(j_3013 + 1));
213.                     listMahasiswa_3013.set(j_3013 + 1,
temp_3013);
214.                 }
215.             }
216.
217.             areaProses_3013.append("Pass " + (i_3013 + 1) +
" : " + tampilNama_3013() + "\n");
218.         }
219.     }
220.
221.     public String tampilNama_3013() {
222.         String hasil_3013 = "[";
223.
224.         for (int i_3013 = 0; i_3013 <
listMahasiswa_3013.size(); i_3013++) {
225.
226.             hasil_3013 +=
listMahasiswa_3013.get(i_3013).getNama_3013();
227.
228.             if (i_3013 != listMahasiswa_3013.size() - 1) {
229.                 hasil_3013 += ", ";
230.             }
231.         }
232.
233.         hasil_3013 += "]";
234.
235.         return hasil_3013;
236.     }

```

```

237.
238.     public static void main(String[] args) {
239.         new SortingGUI_2511533013().setVisible(true);
240.     }
241. }
242.

```

OUTPUT:

Pada Insertion Sort, data disisipkan ke posisi yang benar satu per satu. Proses sorting ditampilkan dalam bentuk langkah-langkah sehingga pengguna dapat melihat perubahan data setiap proses berlangsung.

Pada Selection Sort, program mencari nama yang paling kecil secara alfabetis lalu menukarnya dengan posisi awal. Hasil sorting menunjukkan data mahasiswa berhasil terurut dari A-Z.

Sorting Mahasiswa GUI

Nama

NIM

Program Studi

Bubble Sort

Nama	NIM	Program Studi
Abdul	3002	Informatika
Annafi	3013	Informatika
Fahri	1015	Informatika
Fathan	7001	Informatika
Rafi	3012	Informatika

=== BUBBLE SORT ===

Pass 1 : [Abdul, Annafi, Fahri, Fathan, Rafi]

Pass 2 : [Abdul, Annafi, Fahri, Fathan, Rafi]

Pass 3 : [Abdul, Annafi, Fahri, Fathan, Rafi]

Pass 4 : [Abdul, Annafi, Fahri, Fathan, Rafi]

Tambah Data Hapus Data Mulai Sorting

Pada Bubble Sort, data dibandingkan satu per satu dengan data di sebelahnya. Jika urutannya salah maka data ditukar. Proses dilakukan berulang hingga semua data terurut dengan benar.

BAB III

KESIMPULAN

3.1 Ringkasan

Berdasarkan praktikum yang telah dilakukan, implementasi Double Linked List dan Queue menggunakan bahasa pemrograman Java berhasil dijalankan dengan baik. Double Linked List mampu melakukan operasi penambahan, penghapusan, dan penelusuran data secara fleksibel karena memiliki pointer next dan prev.

Selain itu, Queue berhasil diimplementasikan menggunakan prinsip FIFO (First In First Out). Operasi enqueue, dequeue, display, isEmpty, isFull, dan reverse dapat berjalan sesuai dengan yang diharapkan.

Melalui praktikum ini, pemahaman mengenai pengelolaan data menggunakan struktur data menjadi lebih baik. Praktikum juga membantu meningkatkan kemampuan dalam menyusun logika pemrograman secara sistematis dan terstruktur.

3.2 Saran

Bagi pengajar, diharapkan dapat memberikan lebih banyak contoh kasus yang relevan dengan kehidupan nyata sehingga mahasiswa dapat lebih mudah memahami penerapan struktur data dalam dunia kerja.

DAFTAR PUSTAKA

1. Kadir, Abdul. 2018. Dasar Pemrograman Java. Yogyakarta: Andi Offset.
2. Nugroho, Adi. 2019. Struktur Data dengan Java. Bandung: Informatika.
3. Sedgewick, Robert dan Kevin Wayne. 2011. Algorithms Fourth Edition. Boston: Addison-Wesley.
4. Oracle Corporation. 2026. Java Documentation. <https://docs.oracle.com/javase/>
5. Weiss, Mark Allen. 2014. Data Structures and Algorithm Analysis in Java. United States: Pearson Education.